

NG Marshal — Software Brief

Project spec & build reference. Single source of truth for what the tool is, how the data flows, and the build order.

Product: ITV incentive tracking + live turnaround (TAT) for container-yard internal transfer vehicles. **Site:** Mundra EXIM. **Repo:** `tGainR/NG-Marshall`. **Stack:** Next.js (App Router) · TypeScript · Tailwind · React reducer/context store · Capacitor (Android APK). **Status:** Phase 1 shipped (TAT spine live off the transport report). Phases 2–4 below.

North star. The tool exists to do two things well: **(1) track ITV incentive**, and **(2) track ITV turnaround (TAT) live**. Planning, dashboards and reporting are layers on top. **TAT is the spine** — every dashboard is a lens on one cycle clock.

1. The four SLA TATs (lead with these)

SLA TAT	START → END	PRIORITY	BASELINE TARGET
Import	Pendency ready → Grounded at yard	—	≈ 91 min
Export	Order → Export delivered	Before cut-off	≈ 152 min
Check Package — On-berth vessel	Order → Back at yard	Highest (urgent)	≈ 185 min
Check Package — Normal	Order → Back at yard	Normal	≈ 185 min

On-berth vs Normal share every touch point — they differ *only* in target + priority (a vessel at berth is tight). In code they are one clock; the class just picks which target judges it. Targets are seeded from the Excel study baseline and **editable in Setup** — changing one re-judges all history instantly (breach is computed live, never baked in).

Scanning is separate — it is a movement but **not** an SLA TAT. It forms no TAT cycle and never enters the turnaround numbers.

Implemented as `SlaClass = "import" | "export" | "checkpkg_onberth" | "checkpkg_normal" ( src/lib/types.ts )`.

2. The operational flow the tool models

Morning pendency arrives

- Shift incharge tells each ITV vendor how many vehicles on import vs export
- Vendors allocate (early morning; drivers sleep ~05:00–07:00/08:00, then start)
- FREE-FLOW round trip begins:
 - ITV finishes work → enters terminal → export offloaded → checker loads an export
 - driver takes parchi at EXIM gate → drives to terminal gate → import parchi
 - offload export → pick up import → import back to EXIM → repeat
- After morning allocation the vendor has NO active role except:
 - (1) escalating delays → the SLA / TAT breach board
 - (2) urgent / check-package containers → the < priority override

Notes that shape the model: - **More import than export**, so export pairing runs out → **straight-import** ITVs are common; **straight-export** is rare. - An ITV is allocated to a **location** (terminal/gate), *not* a specific container. The gate assigns the container **FIFO**. So the tool assigns the ITV; it never picks the box. - The round-trip legs the TAT clock measures are **EXIM gate ↔ terminal gate**.

3. Data model

The atom is the **cycle** — one ITV round trip, judged against one SLA target.

```
// src/lib/types.ts
type SlaClass = "import" | "export" | "checkpkg_onberth" | "checkpkg_normal";

interface SlaTarget { targetMin: number; priority: "normal"|"before_cutoff"|"urgent"; atRiskPct: number; }
type SlaConfig = Record<SlaClass, SlaTarget>; // editable in Setup, persisted

interface CycleTat { c: SlaClass; m: number; } // one cycle: its class + turnaround minutes

// Cycles are carried on the day record we already capture:
interface DriverTripLog {
  id; date; driverId?; driverName; itv?; vendor?;
  imp20; imp40; exp20; exp40; scan20; scan40; checkPkg; // move counts (TEU/incentive)
  firstMin?; lastMin?; avgGapMin?; // day timing
  cycles?: CycleTat[]; // ← the TAT spine
  source: "manual" | "upload"; at;
}
```

Assignment (planner) carries the urgent-override fields used by the vendor's post-allocation role:

```
interface Assignment {
  target; purpose; pickup?; commit?: "tentative"|"confirmed"; note?;
  priority?: boolean; customer?; divertedFrom?; by?; at?; // ← s urgent override
}
```

Departed container keeps only **dwellHrs** (its TAT) + volume — the smallest record that answers TAT and pendency history. Retention caps trim oldest first (**RETENTION** in **store.tsx**).

4. Touch points per cycle — and how each is captured (now → later)

The fields never change; only **who fills them** evolves. This is the key to the phased build.

LEG BOUNDARY (TOUCH POINT)	PHASE 1 — NOW	PHASE 2 — PARCHI SCAN	PHASE 3 — AUTO
Order / assigned	shift-incharge allocation	driver app assignment	pendency feed / TOS
EXIM gate-out (export leaves)	— (proxy)	parchi photo + submit	geofence + gate
Terminal gate-in	transport-report gate timestamp	parchi at terminal gate	geofence + gate + TOS
Export offloaded / import parchi	—	parchi photo	TOS event
Terminal gate-out (import leaves)	— (proxy)	parchi submit	geofence + gate
EXIM gate-in / grounded	transport-report gate timestamp	parchi submit	geofence + gate

Phase 1 TAT = gate → **next-gate cycle** (the proxy) computed from the transport report's **TRANSACTION TIME** per truck-per-day. Coarse but real today. Phase 2 replaces it with **precise per-leg** times; the same SLA cards fill in sharper with no rework. Check-package cycles from the transport report count as **Normal** until the driver form's *On-berth* / *Normal* circle labels them.

5. The metrics engine

src/lib/tat.ts — pure functions, the single source every SLA screen reads:

- flattenCycles(logs)** → one row per cycle with **{c, m, itv, vendor, driver, date}**.
- classStats(rows, cfg)** → per class: **n, avg, median, p90, within, breach, breachPct** vs target.
- groupStats(rows, cfg, dim)** → roll up by **itv / vendor / driver / date** (worst-offender tables).
- overall(stats)** → headline totals.
- slaStatusOf(min, target, running)** → **within | at_risk | breach** (at-risk once a *running* cycle passes **atRiskPct** of target).

Related engines already in the codebase: **incentive** (**logIncentive** from the rate card), **allocation** (round-trip paired / straight-import / straight-export in the ITV Planner), **rotation & productivity** (trips/TEU per ITV & driver in Trip Log / Drivers).

6. Incentive ↔ container reconciliation (money rule)

Incentive credits when the driver **submits the parchi** (not on mere photo upload). But a parchi can be submitted for a container the ITV never actually brought in — so every incentive-earning parchi is reconciled against **actual gate-in** (source: the **transport report**, which has gate-in details):

- match found → **Confirmed**
- no match → **Flagged** in a discrepancy report; **incentive is not auto-clawed** — the user reviews.

Chosen model: *provisional, then confirm/flag*. Reversible to *pay-only-on-confirmed* later — same plumbing (the cycle's `containerNo` is the join key). This is Phase 2 work; the reconciliation source (transport report gate-in) is already parsed.

7. Dashboards & alerts

- **SLA / TAT tab** (live): per-class TAT-vs-target cards, breach %, and a *who's-missing-SLA* table sortable by ITV/vendor/driver/date; CSV export. ← **Phase 1, shipped**.
- **ITV Planner**: shift roster (mark live), round-trip work queues, quick/auto allocate, and the ⚡ **priority override** (urgent divert for a customer or check-package; pinned red, never auto-swept, audited).
- **Drivers / Trip Log**: trips, TEU, ₹ incentive; per-ITV cycle-time.
- **Alerts**: SLA thresholds live in `SLaConfig`. Engine flags **at-risk** (running past `atRiskPct`) and **breach** (past target); a leg sitting too long = *stuck*. Fires on transport-report refresh today; **live per-leg once the driver app lands** (Phase 2).

8. Build plan

Phase 1 — TAT spine off the transport report. ✅ **Shipped**. `SLaConfig` + editable targets · per-class cycle extraction · `tat.ts` engine · SLA/TAT tab · breach tables.

Phase 2 — parchi scan + live alerts. Driver assigned to location → photographs & submits parchi (OCR/barcode read of container no.) → precise per-leg touch points → live at-risk / stuck alerts → incentive-on-submit + Confirmed/Flagged reconciliation vs gate-in. *Blocked on*: a sample photo of a real parchi + whether it carries a barcode.

Phase 3 — auto capture. Geofence + gate + TOS + pendency feed fill the same touch points automatically; manual/parchi become fallback + edit.

Phase 4 — VMT in the ITV (end-state). A Vehicle-Mounted Terminal in the cab captures everything with zero driver effort. See §10.

9. Where things live (orientation)

CONCERN	FILE
Domain types, SLA config, helpers	<code>src/lib/types.ts</code>
Parsing (pendency, transport report, masters)	<code>src/lib/importer.ts</code>
TAT engine	<code>src/lib/tat.ts</code>
State + reducer (persist, hydrate, retention)	<code>src/lib/store.tsx</code>
Console shell + tabs	<code>src/app/console/page.tsx</code>
SLA/TAT tab	<code>src/app/console/SlaTab.tsx</code>
ITV Planner (allocation + priority)	<code>src/app/console/ItvPlannerTab.tsx</code>
Drivers / Trip Log	<code>src/app/console/DriversTab.tsx</code> , <code>DriverLogTab.tsx</code>
Driver mobile (parchi capture)	<code>src/app/driver/page.tsx</code>
Backend datastore (local / http / supabase)	<code>src/lib/data/*</code> , <code>src/app/api/state/route.ts</code>

Backend is selected by `NEXT_PUBLIC_BACKEND` (`local` browser · `http` self-host file · `supabase` cloud). Self-host target: Mac mini now → AWS later.

10. VMT in the ITV — the end-state capture layer

Goal: move capture off the driver's phone/parchi and into the vehicle, so the ITV itself reports every detail automatically. Crucially, **the data model does not change** — a VMT is just another *source* filling the same touch points from §4. Nothing downstream (TAT engine, SLA board, incentive, reconciliation) is rebuilt; we only add an ingest path.

What a VMT is: a rugged Android terminal mounted in the cab (the same NG Marshal Capacitor app runs on it), plus telematics sensors. Because it is Android, Phase 2's app *is* the VMT app — the VMT just adds hardware inputs.

How it captures each thing:

DETAIL	HOW THE VMT GETS IT
Gate-in / gate-out per leg (the TAT clock)	GPS geofencing — polygons drawn around EXIM gate and each terminal gate; entry/exit stamps the touch point automatically, no driver tap
Live position / stuck detection	GPS position + speed streamed → live map; no movement inside a zone past its SLA = <i>stuck</i> alert
Container / parchi linkage	on-board barcode / RFID scanner (or the same camera-OCR as the phone app) reads the container number at the gate
Engine hours, idling, ignition	OBD-II / CAN-bus tap → productive vs idle time, fuel/diesel events
Trip legs & rotation	derived from the geofence crossings above — same CycleTat records, now precise and hands-free

Connectivity & sync: a 4G SIM in the VMT; an **offline queue that syncs when signal returns** — the store already uses snapshot + optimistic-lock **rev**, which tolerates intermittent connectivity, so this needs an ingest endpoint (extend **/api/ingest**), not a redesign.

Practical path to get there: 1. **Now:** phone app + parchi (Phase 2) — proves the touch-point capture with real drivers. 2. **Pilot VMT:** fit 3–5 ITVs with an off-the-shelf rugged Android VMT + GPS; enable geofencing to auto-stamp gate crossings; keep the phone/parchi as fallback. Compare VMT gate times vs transport-report gate times to validate. 3. **Add peripherals:** barcode/RFID scanner for container linkage; OBD/CAN for engine data. 4. **Fleet roll-out** once the pilot's TAT numbers match the manual baseline.

Build vs buy: the sensible route is **buy the hardware** (rugged Android VMT + GPS/telematics unit are commodity in fleet ops) and **build only the ingest + geofence config** in NG Marshal. That keeps the whole thing one codebase — phone, VMT and console all read/write the same cycle records.

Right now, this is future work. The active system is the **phone app + parchi scan** (Phase 2); VMT is the target it upgrades into without rework.